

Projekt do predmetu ISA - Filtrujúci DNS Resolver

Filip Botlo - xbotlo01

2. November 2025

Obsah

1	Úvod do problematiky	2
2	Návrh aplikácie	2
2.1	Architektúra systému	2
3	Implementácia	2
3.1	Prehľad tried a ich funkcionality	2
3.1.1	Trieda CLIParser	2
3.1.2	Trieda DNSResolver	3
3.1.3	Trieda DNSProtocol	3
3.2	DNS protokol implementácia	3
3.3	Filtrovacia logika	3
3.4	Rozšírenia aplikácie	4
3.4.1	Verbose mód	4
3.4.2	Štatistiky	4
4	Testovanie	4
4.1	Manuálne testovanie pomocou dig	4
4.1.1	Testovanie A záznamov --povolené domény	4
4.1.2	Testovanie A záznamov – blokové domény	5
4.1.3	Testovanie subdomén	5
4.1.4	Testovanie rôznych typov DNS záznamov	5
4.1.5	Testovanie zachovania Query ID	6
4.1.6	Testovanie IPv6	6
4.2	Automatizované testovanie	6
4.2.1	Python testovací skript	6
4.3	Analýza paketov pomocou Wiresharku	7
4.4	Výsledky testov	7
5	Záver	7
6	Literatúra a zdroje	8

1 Úvod do problematiky

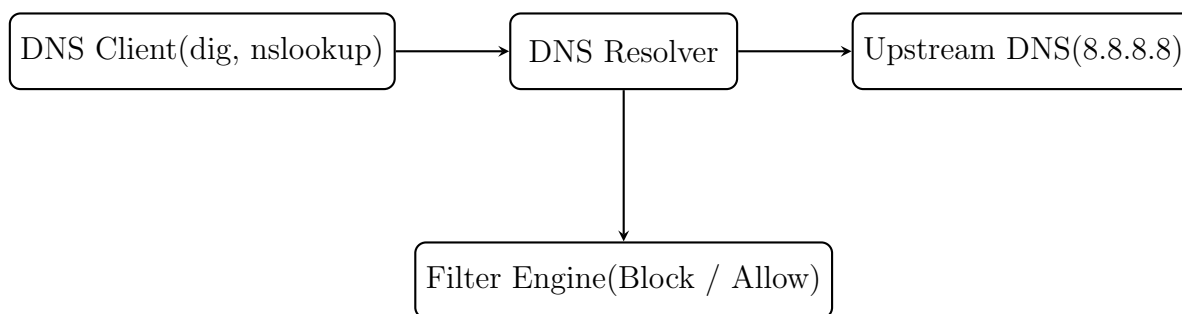
DNS (Domain Name System) je hierarchický distribuovaný systém pre preklad doménových mien na IP adresy a naopak. Je základným stavebným kameňom internetu, ktorý umožňuje používateľom pristupovať k webovým stránkam pomocou ľahko zapamätateľných názvov namiesto číselných IP adries.

Cieľom projektu bolo implementovať vlastný DNS resolver, ktorý spracúva DNS dotazy a aplikuje na ne filtrovaciu logiku.

2 Návrh aplikácie

2.1 Architektúra systému

Aplikácia je navrhnutá ako modulárny systém s jasne oddelenými zodpovednosťami. Používateľ (klient) posielá DNS dotaz napríklad cez nástroje `dig` alebo `nslookup`. Dotaz prichádza do DNS resolvera, ktorý ho analyzuje a rozhodne, či je možné ho odoslať na DNS server (napr. Google DNS 8.8.8.8), alebo ho treba blokovať podľa pravidiel filtra.



Obr. 1: Architektúra systému

3 Implementácia

3.1 Prehľad tried a ich funkcionality

Aplikácia je implementovaná v jazyku C++ s využitím objektovo orientovaného programovania. Hlavné triedy sú navrhnuté tak, aby každá mala jasne definovanú zodpovednosť a minimalizovala závislosti medzi modulmi.

3.1.1 Trieda `CLIParser`

Trieda je zodpovedná za spracovanie argumentov príkazového riadka. Obsahuje metódy pre parsovanie jednotlivých parametrov, validáciu vstupných hodnôt a generovanie nápovedy. Hlavné funkcie tejto triedy zahŕňajú:

- `parse()` – hlavná metóda pre spracovanie argumentov
- `validateArguments()` – kontrola platnosti zadaných parametrov
- `printHelp()` – zobrazenie nápovedy pre používateľa

Trieda podporuje všetky potrebné parametre ako port, upstream DNS server, cestu k filtrovaciemu súboru a verbose mód. Implementuje error handling pre neplatné vstupy.

3.1.2 Trieda DNSResolver

Trieda predstavuje jadro aplikácie a koordinuje všetky operácie. Je zodpovedná za vytvorenie sieťových socketov, implementáciu serverového cyklu a správu komunikácie medzi klientmi a upstream DNS serverom. Kľúčové metódy tejto triedy:

- `createClientSocket()` – vytvorenie socketu pre prijímanie dotazov od klientov
- `createResolverSocket()` – vytvorenie socketu pre komunikáciu s upstream DNS
- `loadFilterFile()` – načítanie zoznamu blokováných domén zo súboru
- `handleQuery()` – spracovanie jednotlivého DNS dotazu
- `run()` – hlavný serverový cyklus

Serverový cyklus používa `select()` s `recvfrom()` pre prijímanie dotazov a `sendto()` pre odosielanie odpovedí.

3.1.3 Trieda DNSProtocol

Trieda `DNSProtocol` obsahuje všetku logiku súvisiacu s DNS protokolom podľa RFC 1035. Je implementovaná ako statická trieda s utility metódami pre prácu s DNS správami. Hlavné funkcie:

- `parseDomain()` – parsovanie doménových mien z DNS správ
- `isQuery()` – kontrola, či je správa DNS dotazom
- `processQuery()` – hlavná logika filtrovania a spracovania dotazov
- `sendErrorResponse()` – generovanie chybových DNS odpovedí
- `getClientAddressString()` – konverzia adresy na čitateľný formát

3.2 DNS protokol implementácia

Parsovanie doménových mien je jednou z najzložitejších častí implementácie. Domény sú uložené ako sekvencia labelov, kde každý label má svoju dĺžku nasledovanú obsahom. Resolver dokáže správne dekodovať tieto štruktúry a rekonštruovať plné doménové meno.

3.3 Filtrovacia logika

Najskôr sa doména normalizuje – prevedie sa na malé písmená a odstráni sa nadbytočné znaky ako koncové bodky. Následne sa kontroluje presná zhoda s blokovými doménami.

Pre podporu subdomén sa implementuje algoritmus, ktorý kontroluje, či normalizovaná doména končí bodkou nasledovanou niektorou z blokováných domén. Toto umožňuje blokovat' všetky poddomény bez potreby explicitne uvádzať každú z nich.

Filtrovacia logika podporuje rôzne typy DNS záznamov, pričom A záznamy sú plne podporované. Ostatné typy záznamov sú odmietnuté s chybovým kódom NOTIMP.

3.4 Rozšírenia aplikácie

3.4.1 Verbose mód

Verbose mód je implementovaný ako voliteľný parameter `-v` alebo `--verbose`. Pri aktivácii tohto módu resolver vypisuje podrobné informácie o každom spracovanom DNS dotaze, čo značne uľahčuje ladenie a monitorovanie činnosti aplikácie.

Implementácia verbose módu je integrovaná do triedy `DNSResolver` a `DNSProtocol`, kde sa kontroluje flag `verbose_` pred každým výpisom diagnostických informácií.

3.4.2 Štatistiky

Rozšírenie štatistík je implementované ako voliteľný parameter `--stats`. Toto rozšírenie umožňuje zbieranie a zobrazenie štatistík o činnosti DNS resolvera na konci jeho behu.

Štatistiky zahŕňajú nasledujúce metriky:

- Celkový počet dotazov
- Blokové dotazy
- Preposlané dotazy
- Ostatné typy dotazov (NOTIMP)
- Presné zhody
- Zhody s poddomenami

Implementácia štatistík využíva štruktúru `Statistics` definovanú v `DNSProtocol.hpp`.

4 Testovanie

Testovanie DNS resolvera bolo navrhnuté tak, aby pokrylo všetky kritické aspekty implementácie a overilo správne správanie systému podľa očakávaní. Testovanie sa skladalo z manuálnych testov pomocou nástroja `dig` a automatizovaných testov pomocou Python skriptu.

4.1 Manuálne testovanie pomocou dig

Nástroj `dig` bol zvolený ako hlavný nástroj pre testovanie, pretože ide o štandardnú implementáciu DNS klienta, ktorá správne generuje DNS dotazy podľa RFC 1035. Pre každý test som definoval jasné očakávania na základe implementácie resolvera.

4.1.1 Testovanie A záznamov – povolené domény

Pri testovaní povolených domén som očakával, že resolver:

- Prijme dotaz a správne ho preposiela na upstream DNS server
- Zachová pôvodné Query ID, aby odpoveď zodpovedala pôvodnému dotazu
- Vráti správnu odpoveď so statusom `NOERROR`

- Obsah odpovede bude obsahovať IP adresu z upstream DNS servera

Príklad testu:

```
dig @127.0.0.1 -p 5353 google.com
```

Pomocou tohto dotazu som overil, že doména `google.com`, ktorá nie je v zozname blokovaných domén, je správne preposlaná na resolver `8.8.8.8`. Očakával som návrat správnej IP adresy a status kódu `NOERROR`. Verbose mód mi umožnil sledovať celý proces – od prijatia dotazu cez preposlanie na upstream DNS až po odoslanie odpovede klientovi.

4.1.2 Testovanie A záznamov – blokované domény

Pri testovaní blokovaných domén som očakával, že resolver:

- Rozpozná doménu v zozname blokovaných domén (presná zhoda alebo subdoména)
- Nezasiela dotaz na upstream DNS server
- Vytvorí chybovú odpoveď so statusom `REFUSED` (kód 5)
- Zachová Query ID z pôvodného dotazu

Príklad testu:

```
dig @127.0.0.1 -p 5353 facebook.com
```

Doména `facebook.com` je explicitne uvedená v súbore `blocked_domains.txt`, preto som očakával, že dotaz bude odmietnutý s kódom `REFUSED`. Overil som, že resolver správne identifikoval presnú zhodu a nezaslal dotaz na upstream DNS server, čím sa potvrdila správna funkčnosť filtrovacej logiky.

4.1.3 Testovanie subdomén

Kľúčová funkčnosť resolvera je schopnosť blokovat' aj subdomény blokovaných domén. Pre test som použil subdoménu `www.facebook.com`, pričom som očakával, že bude blokována, pretože `facebook.com` je v zozname blokovaných domén.

```
dig @127.0.0.1 -p 5353 www.facebook.com
```

Očakával som status kód `REFUSED`, čo potvrdilo, že implementácia správne kontroluje, či dotazovaná doména končí bodkou nasledovanou niektorou z blokovaných domén.

4.1.4 Testovanie rôznych typov DNS záznamov

Podľa implementácie resolver podporuje len A záznamy (typ 1). Pre ostatné typy záznamov som očakával status kód `NOTIMP` (kód 4).

Test MX záznamov:

```
dig @127.0.0.1 -p 5353 google.com MX
```

Očakával som status `NOTIMP`, pretože MX záznamy (typ 15) nie sú podporované. Rovnaké očakávanie som mal aj pre AAAA záznamy (typ 28), NS, TXT, CNAME a ďalšie typy záznamov.

4.1.5 Testovanie zachovania Query ID

Dôležitým aspektom DNS protokolu je zachovanie Query ID v odpovedi. V aplikácii nemanimulujem s Query ID, preto som očakával, že pôvodné Query ID z dotazu bude zachované v odpovedi, čo je kritické pre správne spárovanie odpovede s pôvodným dotazom klientom.

Pre overenie som porovnal Query ID v odoslanom dotaze (viditeľné v verbose výstupe) s Query ID v odpovedi získanej cez `dig`, čím som potvrdil, že implementácia správne zachováva Query ID bez modifikácie.

4.1.6 Testovanie IPv6

Overil som funkčnosť resolvera aj s IPv6 adresami:

```
dig @::1 -p 5353 google.com A
```

Očakával som, že resolver správne prijme dotaz cez IPv6 socket a následne ho prepošle na upstream DNS server (8.8.8.8) cez IPv4. Tento test overil správnu implementáciu dual-stack podpory.

4.2 Automatizované testovanie

Pre komplexnejšie testovanie som vytvoril Python testovací skript `simple_test.py`, ktorý automatizuje opakujúce sa testy a umožňuje systematické overenie funkcionality.

4.2.1 Python testovací skript

Testovací skript `simple_test.py` pokrýva nasledujúce kategórie testov:

- **Kompilácia a základné funkcie** – overenie úspešnej kompilácie a zobrazenia nápovedy
- **Validácia argumentov** – testovanie rôznych kombinácií argumentov, poradia parametrov, neplatných hodnôt
- **IPv4/IPv6 podpora** – testovanie s rôznymi typmi adries a doménovými menami DNS serverov
- **Spracovanie súborov** – testovanie rôznych formátov koncov riadkov (Linux, Windows, Mac), komentárov, prázdnych riadkov
- **DNS dotazy** – systematické testovanie blokových a povolených domén, subdomén, case sensitivity
- **Rôzne typy dotazov** – overenie správneho odmietania nepodporovaných typov (MX, AAAA, TXT, NS, CNAME, atď.)
- **Chybové scenáre** – testovanie neexistujúcich súborov, neplatných DNS serverov, timeoutov

Skript automaticky spúšťa DNS resolver v pozadí, vykonáva testy a následne vyhodnotí výsledky. Poskytuje detailné štatistiky o úspešnosti testov a identifikuje zlyhané testy s popisom chýb.

4.3 Analýza paketov pomocou Wiresharku

Pri ladení a overovaní správneho formátu DNS správ som využíval Wireshark na zachytenie a analýzu DNS paketov. Wireshark som použil najmä na:

- **Overenie formátu DNS hlavičky** – kontrola, či sú všetky polia v DNS hlavičke správne nastavené podľa RFC 1035
- **Kontrola Query ID** – overenie, že Query ID v odpovedi zodpovedá Query ID v dotaze
- **Analýza response kódov** – kontrola, či sú response kódy (NOERROR, REFUSED, NOTIMP) správne nastavené v DNS odpovediach
- **Overenie štruktúry paketov** – kontrola, či DNS pakety majú správnu štruktúru (hlavička, question sekcia, answer sekcia)

Wireshark som nevyužíval len na pasívne sledovanie, ale na aktívne hľadanie špecifických patternov v paketoch, ako napríklad správne nastavenie bitov v DNS hlavičke alebo zachovanie Query ID pri preposielaní dotazov na upstream DNS server.

4.4 Výsledky testov

Všetky testy boli navrhnuté na základe konkrétnych očakávaní založených na implementácii resolvera. Testovanie potvrdilo:

- Správne filtrovanie blokových domén (presné zhody aj subdomény)
- Správne preposielanie povolených domén na upstream DNS server
- Zachovanie Query ID v odpovediach
- Správne odmietanie nepodporovaných typov záznamov (NOTIMP)
- Podpora IPv4 aj IPv6 klientov
- Správne spracovanie rôznych formátov filtrovacieho súboru
- Case-insensitive porovnávanie domén

Automatizovaný testovací skript úspešne prešiel 92 testov, čo potvrdilo správnu implementáciu všetkých požadovaných funkcií.

5 Záver

Vďaka projektu som sa naučil pracovať s DNS protokolom na nízkej úrovni a lepšie pochopil jeho štruktúru podľa RFC 1035. Získal som praktické skúsenosti so sieťovým programovaním v C++ a s implementáciou vlastného resolvera.

6 Literatúra a zdroje

1. <https://tools.ietf.org/html/rfc1035>
2. <https://man7.org/linux/man-pages/man2/socket.2.html>
3. <https://man7.org/linux/man-pages/man2/sendto.2.html>
4. <https://man7.org/linux/man-pages/man2/recvfrom.2.html>
5. <https://man7.org/linux/man-pages/man2/select.2.html>
6. <https://man7.org/linux/man-pages/man7/ipv6.7.html>